

Module 3: Querying and Global Secondary Indexes

 web.archive.org/web/20230608201211/https://aws.amazon.com/getting-started/hands-on/create-manage-nonrelational-database-dynamodb/module-3

Getting Started with AWS

Create and Manage a Nonrelational Database

with Amazon DynamoDB

You will walk through some simple examples of retrieving multiple items in one API call with DynamoDB

Overview

In this module, you walk through some simple examples of retrieving multiple items in one API call with DynamoDB. You also learn how to use secondary indexes to enable additional query patterns on your DynamoDB tables.

Application use cases

In Module 2, you learned how to retrieve a single book from a DynamoDB table by using the *GetItem* API call. This access pattern is useful, but your application also needs to be able to retrieve multiple items in one call. For example, you may want to retrieve all books that were written by John Grisham so that you can display them to users. In Step 1 in this module, you use the *Query* API to retrieve all books by a specific author.

Both the *GetItem* API call to get a single book and the *Query* API call to retrieve all books by an author use the specified primary key on your *Books* table. However, you may want to enable additional access patterns, such as retrieving all books in a particular category such as history or biography. *Category* is not part of your table's primary key, but you can create a secondary index to allow for additional access patterns. You will create a secondary index and query the secondary index in Steps 2 and 3 of this module.

Time to complete

15 minutes

Implementation

- Retrieve multiple items with a query

When your table uses a composite primary key, you can retrieve all items with the same hash key by using the *Query* API call. For your application, this means you can retrieve all books with the same *Author* attribute.

In the AWS Cloud9 terminal, run the following command.

```
$ python query_items.py
```

This command runs the following script that retrieves all books written by John Grisham.

```
import boto3
from boto3.dynamodb.conditions import Key

# boto3 is the AWS SDK library for Python.
# The "resources" interface allows for a higher-level abstraction than the low-level clien
# For more details, go to http://boto3.readthedocs.io/en/latest/guide/resources.html
dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
table = dynamodb.Table('Books')

# When making a Query API call, you use the KeyConditionExpression parameter to specify th
# You're using the Key object from the Boto 3 library to specify that you want the attribu
# to equal "John Grisham" by using the ".eq()" method.
resp = table.query(KeyConditionExpression=Key('Author').eq('John Grisham'))

print("The query returned the following items:")
for item in resp['Items']:
    print(item)
```

After you run the script, you should see two John Grisham books, *The Firm* and *The Rainmaker*.

```
$ python query_items.py
The query returned the following items:
{'Title': 'The Firm', 'Formats': {'Hardcover': 'Q7QWE3U2', 'Paperback': 'ZVZAYY4F', 'Audio
{'Title': 'The Rainmaker', 'Formats': {'Hardcover': 'J4SUKVGU', 'Paperback': 'D7YF4FCX'},
```

Retrieving multiple items with a single call in DynamoDB is a common pattern and easy to do with the *Query* API call.

- Creating a secondary index

DynamoDB allows you to create secondary indexes to account for additional data access patterns on your table. Secondary indexes are a powerful way to add query flexibility to a DynamoDB table.

DynamoDB has two kinds of secondary indexes: global secondary indexes and local secondary indexes. In this section, you add a global secondary index to your *Category* attribute that will allow you to retrieve all books in a particular category.

The following example script adds a global secondary index to an existing table.

```
import boto3

# Boto3 is the AWS SDK library for Python.
# You can use the low-level client to make API calls to DynamoDB.
client = boto3.client('dynamodb', region_name='us-east-1')

try:
    resp = client.update_table(
        TableName="Books",
        # Any attributes used in your new global secondary index must be declared in Attri
        AttributeDefinitions=[
            {
                "AttributeName": "Category",
                "AttributeType": "S"
            },
        ],
        # This is where you add, update, or delete any global secondary indexes on your ta
        GlobalSecondaryIndexUpdates=[
            {
                "Create": {
                    # You need to name your index and specifically refer to it when using
                    "IndexName": "CategoryIndex",
                    # Like the table itself, you need to specify the key schema for an ind
                    # For a global secondary index, you can use a simple or composite key
                    "KeySchema": [
                        {
                            "AttributeName": "Category",
                            "KeyType": "HASH"
                        }
                    ],
                    # You can choose to copy only specific attributes from the original it
                    # You might want to copy only a few attributes to save space.
                    "Projection": {
                        "ProjectionType": "ALL"
                    },
                    # Global secondary indexes have read and write capacity separate from
                    "ProvisionedThroughput": {
                        "ReadCapacityUnits": 1,
                        "WriteCapacityUnits": 1,
                    }
                }
            }
        ]
    )
```

```

        }
    ],
)
print("Secondary index added!")
except Exception as e:
    print("Error updating table:")
    print(e)

```

Creating a global secondary index has a lot in common with creating a table. You specify a name for the index, the attributes that will be in the index, the key schema of the index, and the provisioned throughput (the maximum capacity an application can consume from a table or index). Provisioned throughput on each index is separate from the provisioned throughput on a table. This allows you to define throughput granularly to meet your application's needs.

Run the following command in your terminal to add your global secondary index.

```
$ python add_secondary_index.py
```

This script adds a global secondary index called *CategoryIndex* to your *Books* table.

- Querying a secondary index

Now that you have the *CategoryIndex*, you can use it to retrieve all books with a particular category. Using a secondary index to query a table is similar to using the *Query* API call. You now add the index name to the API call.

When you add a global secondary index to an existing table, DynamoDB asynchronously backfills the index with the existing items in the table. The index is available to query after all items have been backfilled. The time to backfill varies based on the size of the table.

You can use the *query_with_index.py* script to query against the new index. Run the script in your terminal with the following command.

```
$ python query_with_index.py
```

That command runs the following script to retrieve all books in the store that have the *Category* of *Suspense*.

```
import time

import boto3
from boto3.dynamodb.conditions import Key

# Boto3 is the AWS SDK library for Python.
# The "resources" interface allows for a higher-level abstraction than the low-level clien
# For more details, go to http://boto3.readthedocs.io/en/latest/guide/resources.html
dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
table = dynamodb.Table('Books')

# When adding a global secondary index to an existing table, you cannot query the index un
# This portion of the script waits until the index is in the "ACTIVE" status, indicating i
while True:
    if not table.global_secondary_indexes or table.global_secondary_indexes[0]['IndexStatu
        print('Waiting for index to backfill...')
        time.sleep(5)
        table.reload()
    else:
        break

# When making a Query call, you use the KeyConditionExpression parameter to specify the ha
# If you want to use a specific index, you also need to pass the IndexName in our API call
resp = table.query(
    # Add the name of the index you want to use in your query.
    IndexName="CategoryIndex",
    KeyConditionExpression=Key('Category').eq('Suspense'),
)

print("The query returned the following items:")
for item in resp['Items']:
    print(item)
```

Note that there is a portion of the script that waits until the index is available for querying.

You should see the following output in your terminal.

```
$ python query_with_index.py
```

The query returned the following items:

```
{'Title': 'The Firm', 'Formats': {'Hardcover': 'Q7QWE3U2', 'Paperback': 'ZVZAYY4F', 'Audio': 'D7YF4FCX'},  
{ 'Title': 'The Rainmaker', 'Formats': {'Hardcover': 'J4SUKVGU', 'Paperback': 'D7YF4FCX'},  
{ 'Title': 'Along Came a Spider', 'Formats': {'Hardcover': 'C9NR6RJ7', 'Paperback': '37JVGD
```

The query returns three books by two different authors. This is a query pattern that would have been difficult with your table's main key schema but is easy to implement with the power of secondary indexes.

In the next module, you learn how to update the attributes of an existing item in a table by using the *UpdateItem* API.